

АННОТАЦИЯ

Данная тема посвящена программным средствам решения задач анализа данных. В работе рассматриваются базовые модели машинного обучения и нейронных сетей для прикладного анализа данных. Описываются методы и алгоритмы, применяемые для работы с данными.

В рамках работы были разработаны базовые модели машинного обучения, включая логистическую регрессию, кластеризацию и радиально-базисную нейронную сеть. Эти модели были успешно применены к различным задачам анализа данных, показав свою эффективность и применимость в различных областях.

Кроме того, в рамках работы были разработаны и исследованы нейросетевые модели анализа данных, реализующие векторные представления изображений и слов. Эти модели демонстрируют высокую точность и способность извлекать семантические признаки из данных.

СОДЕРЖАНИЕ

АННОТАЦИЯ.....	3
ВВЕДЕНИЕ.....	5
1 РАЗРАБОТКА БАЗОВЫХ МОДЕЛЕЙ МАШИННОГО ОБУЧЕНИЯ ДЛЯ АНАЛИЗА ДАННЫХ.....	6
1.1 Логистическая регрессия.....	6
1.2 Кластеризация.....	9
1.3 Радиально-базисная нейронная сеть	15
2 НЕЙРОСЕТЕВЫЕ МОДЕЛИ АНАЛИЗА ДАННЫХ	20
2.1 Векторные представления изображений	20
2.2 Векторные представления слов	21
ЗАКЛЮЧЕНИЕ	26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	27
ПРИЛОЖЕНИЯ.....	28

ВВЕДЕНИЕ

Современные технологии искусственного интеллекта играют ключевую роль в решении множества прикладных задач, связанных с анализом данных. Разнообразие подходов и методов, используемых в области искусственного интеллекта, позволяет эффективно обрабатывать большие объемы данных, извлекать из них полезную информацию и принимать обоснованные решения. Данная работа посвящена разработке программных средств для решения прикладных задач ИИ, ориентированных на анализ данных с использованием различных моделей машинного обучения и нейросетевых методов.

Целью данной работы является создание и исследование базовых моделей машинного обучения, таких как логистическая регрессия, кластеризация различными методами, радиально-базисные нейронные сети, а также разработка нейросетевых моделей для работы с векторными представлениями изображений и слов.

1 РАЗРАБОТКА БАЗОВЫХ МОДЕЛЕЙ МАШИННОГО ОБУЧЕНИЯ ДЛЯ АНАЛИЗА ДАННЫХ

Базовые модели машинного обучения являются основой для более сложных и специализированных моделей. Они представляют собой простые и понятные алгоритмы, которые могут быть легко интерпретированы и использованы для решения широкого спектра задач.

Целью данного раздела является изучение базовых моделей машинного обучения, их принципов работы. Будут рассмотрены логистическая регрессия, кластеризация тремя различными методами и радиально-базисная нейронная сеть.

1.1 Логистическая регрессия

Логистическая регрессия – это статистический метод, используемый для бинарной классификации. Она позволяет моделировать вероятность принадлежности наблюдения к одному из двух классов, используя набор объясняющих переменных. Логистическая регрессия основывается на логистической функции, также известной как функция сигмоиды, которая преобразует линейную комбинацию входных переменных в значение между 0 и 1, интерпретируемое как вероятность события.

Рассмотрим работу логистической регрессии на примере распознавания рукописных цифр MNIST, представленных как 64-мерные векторы (Рис. 1.1.1). Набор данных состоит из 1797 примеров. Разделим данные на обучающую и тестовую часть, оставив на тест 30% данных.

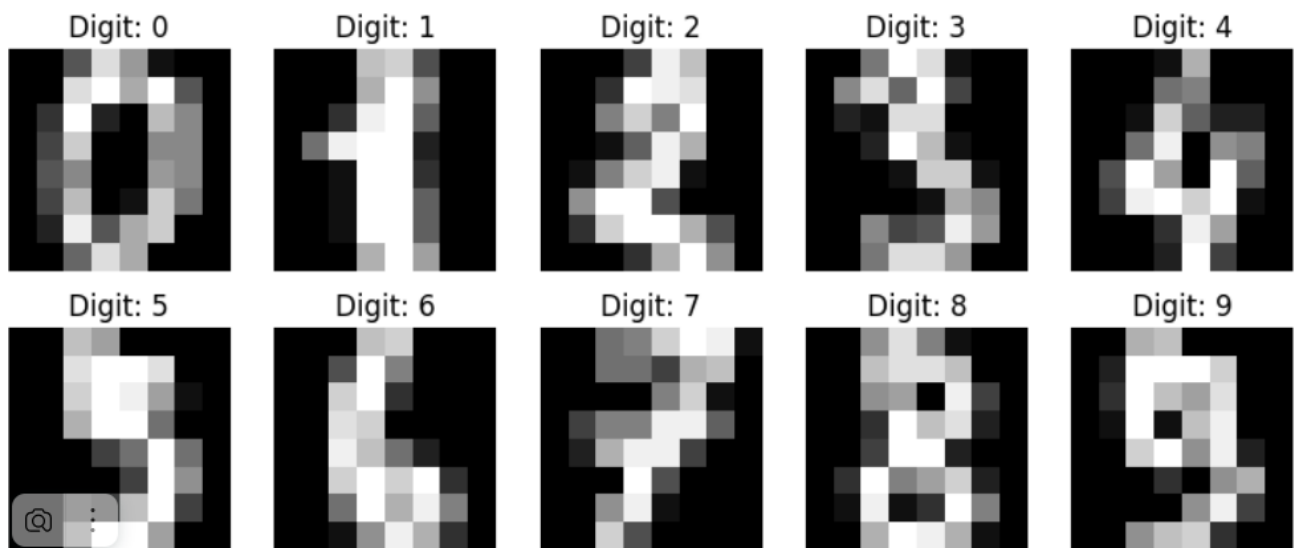


Рисунок 1.1.1 – Набор рукописных цифр MNIST

Создадим модель логистической регрессии из библиотеки `sklearn` для распознавания этих цифр. Будем использовать стандартные (заранее заданные) гиперпараметры, за исключением максимального количества итераций – это значение установим на 10000.

Обучим модель на тренировочных данных и посчитаем ошибку на тестовом наборе, а именно матрицу потерь, точность (accuracy), полноту (recall) и F1 меру (Рис. 1.1.2).

```
Confusion Matrix:
[[53  0  0  0  0  0  0  0  0  0]
 [ 0 47  1  0  0  0  0  0  2  0]
 [ 0  0 47  0  0  0  0  0  0  0]
 [ 0  0  1 52  0  1  0  0  0  0]
 [ 0  1  0  0 59  0  0  0  0  0]
 [ 0  1  0  0  0 63  1  0  0  1]
 [ 0  0  0  0  0  1 52  0  0  0]
 [ 0  0  0  0  0  1  0 54  0  0]
 [ 0  0  0  0  0  1  0  0 42  0]
 [ 0  0  0  1  0  0  0  0  2 56]]
Accuracy: 0.9722222222222222
Recall: 0.9729688736551022
F1 мера: 0.9720884522335643
```

Рисунок 1.1.2 – Ошибка на тестовом наборе данных

Из Рисунка 1.1.2 видно, что модель обладает высокой точностью классификации цифр по 64 компонентам.

Методом главных компонент спроецируем 64-мерные вектора на меньшее пространство N , где $N = \{2, 5, 10, 20, 30, 40, 50, 60\}$, затем повторим процесс обучения и тестирования модели на обработанных данных. Результаты внесем в таблицу для более удобного сравнения (Табл. 1.1.1).

Таблица 1.1.1 – Результаты обучения логистической регрессии

N	Accuracy	Recall	F1_Score	Training_Time
2.0	0.631481	0.634911	0.619639	0.314622
5.0	0.846296	0.840960	0.840126	0.485509
10.0	0.912963	0.910820	0.909155	0.751367
20.0	0.946296	0.948727	0.946950	0.752833
30.0	0.964815	0.963582	0.963462	0.551571
40.0	0.971074	0.971664	0.971045	0.444640
50.0	0.972222	0.972969	0.972088	0.449342
60.0	0.972222	0.972969	0.972088	0.449927

По Таблице 1.1.1 видно, что с увеличением числа главных компонент точность модели возрастает. Однако начиная с $N = 40$ уменьшение ошибки почти незаметно, из чего можно сделать вывод, что $N = 40$ является наиболее оптимальным значением.

Таким образом, логистическая регрессия показала, что она хорошо справляется с задачей классификации по 10 классам. Проведя некоторые эксперименты с размерностью входных данных, мы определили наиболее оптимальные настройки для обучения логистической регрессии. Полный код программы представлен в Приложении А.

1.2 Кластеризация

Кластеризация представляет собой метод группировки объектов таким образом, чтобы объекты внутри одной группы (кластера) были более схожими между собой, чем с объектами из других групп. Этот метод незаменим при анализе неразмеченных данных, когда необходимо выявить скрытые структуры или сегменты в наборе данных. Существуют различные алгоритмы кластеризации, в данном разделе мы рассмотрим методы k-средних, DBSCAN и Gaussian mixtures.

К-средних (K-means) — это один из самых популярных и простых методов кластеризации. Он основан на разбиении набора данных на K кластеров, где K задается пользователем. Алгоритм работает следующим образом:

1. Инициализация: случайным образом выбираются K центроидов (центральных точек кластеров).

2. Назначение точек: каждая точка данных назначается ближайшему центроиду.

3. Обновление центроидов: центроиды пересчитываются как среднее всех точек, принадлежащих соответствующему кластеру.

4. Повторение: шаги 2 и 3 повторяются до тех пор, пока центроиды не перестанут существенно изменяться или не достигнется максимальное количество итераций.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) — метод кластеризации, основанный на плотности точек в пространстве данных. Алгоритм работает следующим образом:

1. Основные точки (Core Points): точки, вокруг которых в пределах радиуса ϵ находится как минимум MinPts других точек.

2. Граничные точки (Border Points): точки, находящиеся в пределах радиуса ϵ от основной точки, но не являющиеся основными.

3. Шумовые точки (Noise Points): точки, которые не являются ни основными, ни граничными.

Алгоритм идентифицирует кластеры, находя группы близко расположенных основных точек и объединяя их с граничными точками.

Метод Gaussian Mixtures (Смесь Гауссовых распределений) использует статистическую модель смеси гауссовых распределений для представления кластеров. Он основан на предположении, что данные являются результатом нескольких гауссовых распределений с разными параметрами. Алгоритм часто применяется с использованием EM (Expectation-Maximization) алгоритма:

1. Инициализация: случайное задание параметров гауссовых распределений (средние, ковариационные матрицы и веса).

2. Шаг ожидания (E-step): вычисление вероятности принадлежности каждой точки к каждому гауссову распределению.

3. Шаг максимизации (M-step): обновление параметров гауссовых распределений, максимизируя вероятность наблюдаемых данных.

Приступим к сравнению алгоритмов. На плоскости создадим три набора данных для кластеризации. Каждый кластер выделен собственным цветом (Рис. 1.2.1).

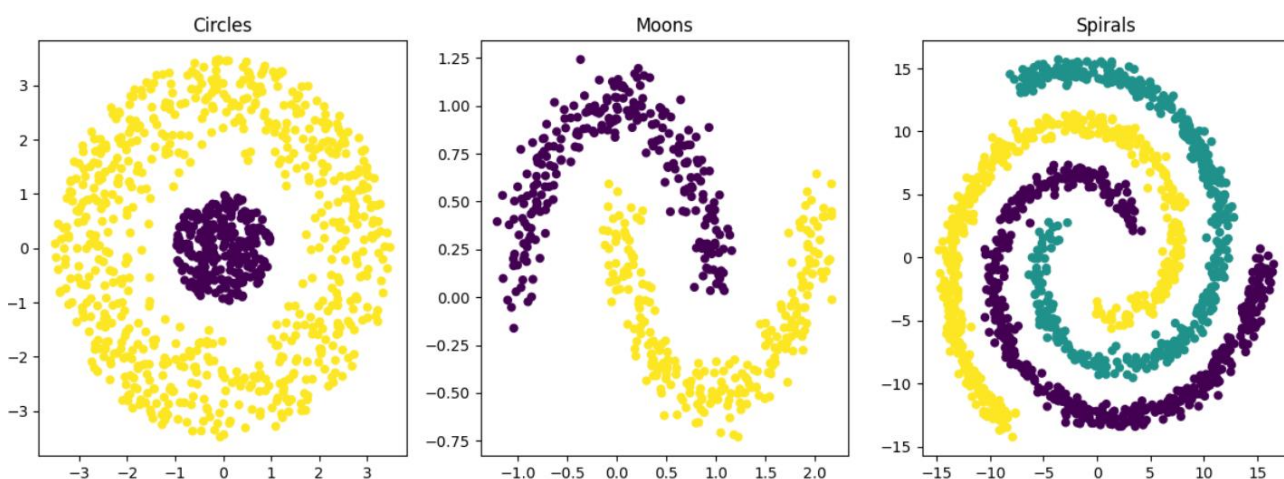


Рисунок 1.2.1 – Наборы данных на плоскости

Проведем кластеризацию представленными ранее методами и посчитаем Rand Index каждого.

В результате работы алгоритма К-средних для набора данных Circles Rand Index равняется -0.00114, для набора Moons RI = 0.265, для набора Spirals RI = 0.0566. Метод плохо справился с задачей из-за специфики алгоритма, не

позволяющего корректно работать с кластерами неправильной формы или разного размера (Рис. 1.2.2, 1.2.3, 1.2.4).

Rand Index для K-means: -0.001144118332334465

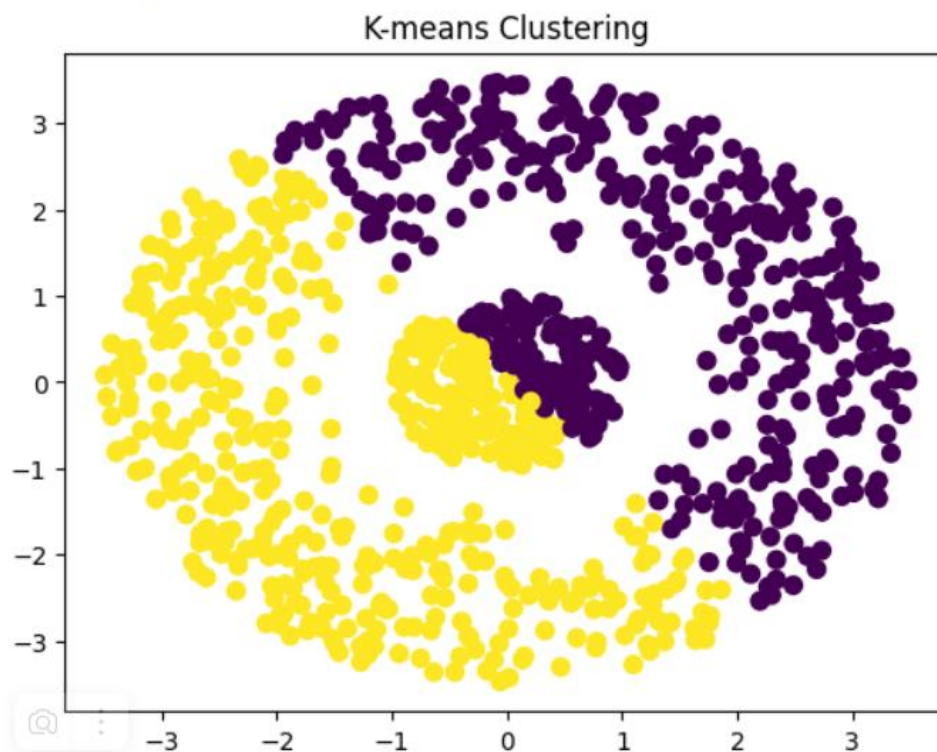


Рисунок 1.2.2 – К-средних для набора Circles

Rand Index для K-means: 0.2647828310668119

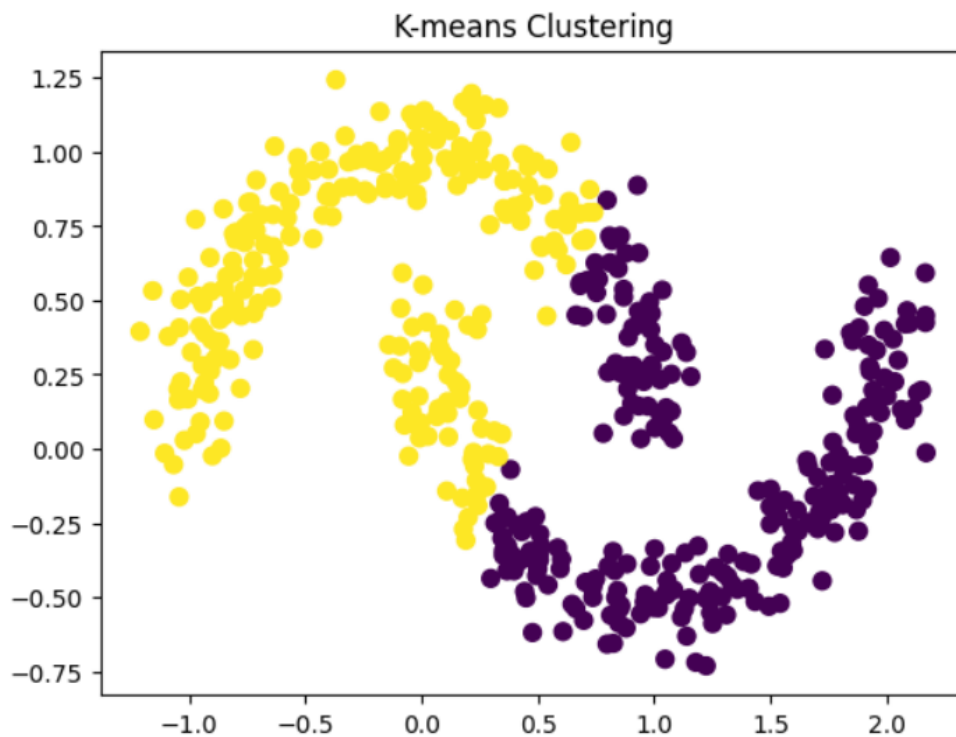


Рисунок 1.2.3 – К-средних для набора Moons

Rand Index для K-means: 0.05661377932704073

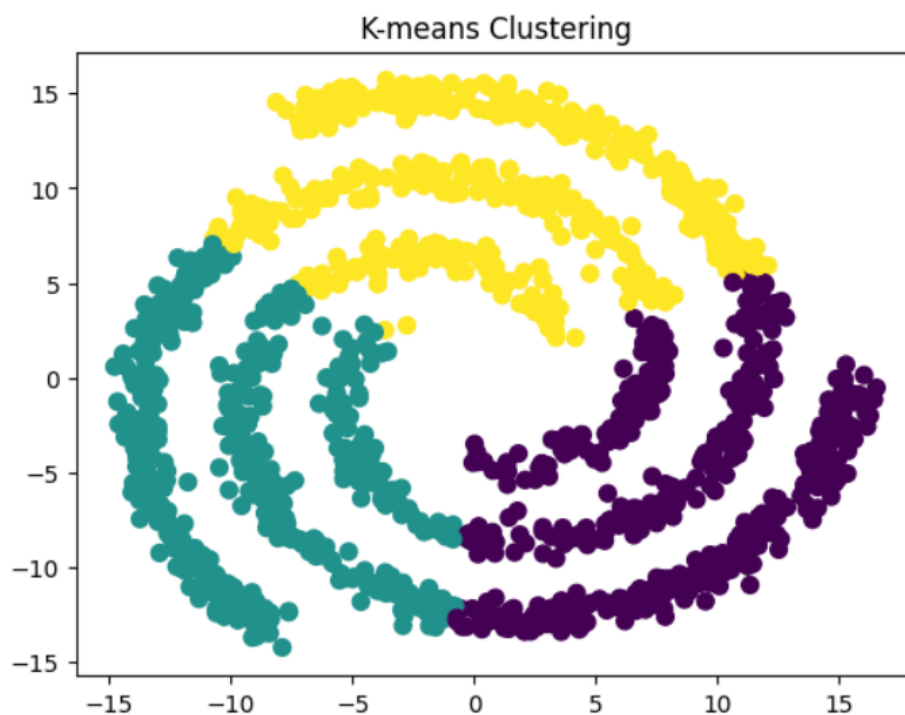


Рисунок 1.2.4 – К-средних для набора Spirals

В результате работы алгоритма DBSCAN для наборов данных Circles, Moons, Spirals Rand Index равняется 1, 1 и 0.998 соответственно. Это идеальный результат, метод полностью справился с задачей (Рис. 1.2.5, 1.2.6, 1.2.7)

Rand Index для DBSCAN: 1.0

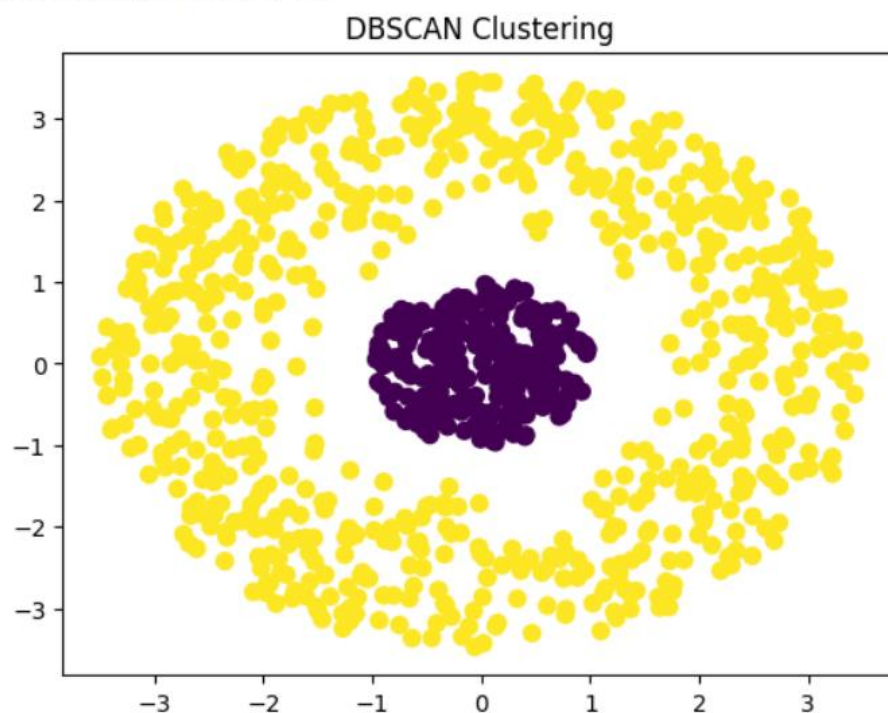


Рисунок 1.2.5 – DBSCAN для набора Circles

Rand Index для DBSCAN: 1.0

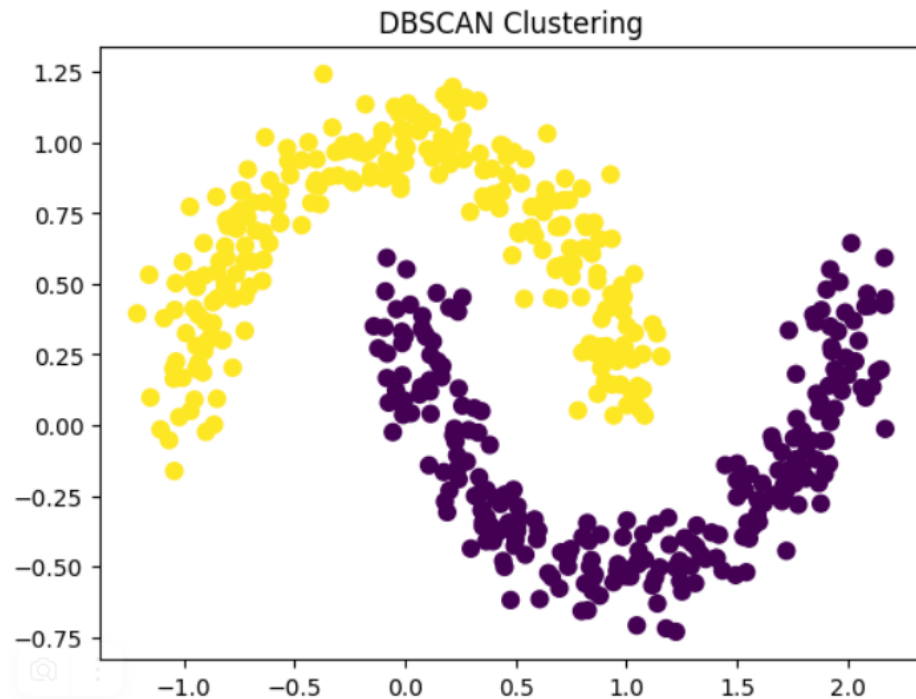


Рисунок 1.2.6 – DBSCAN для набора Moons

Rand Index для DBSCAN: 0.9979993320004471

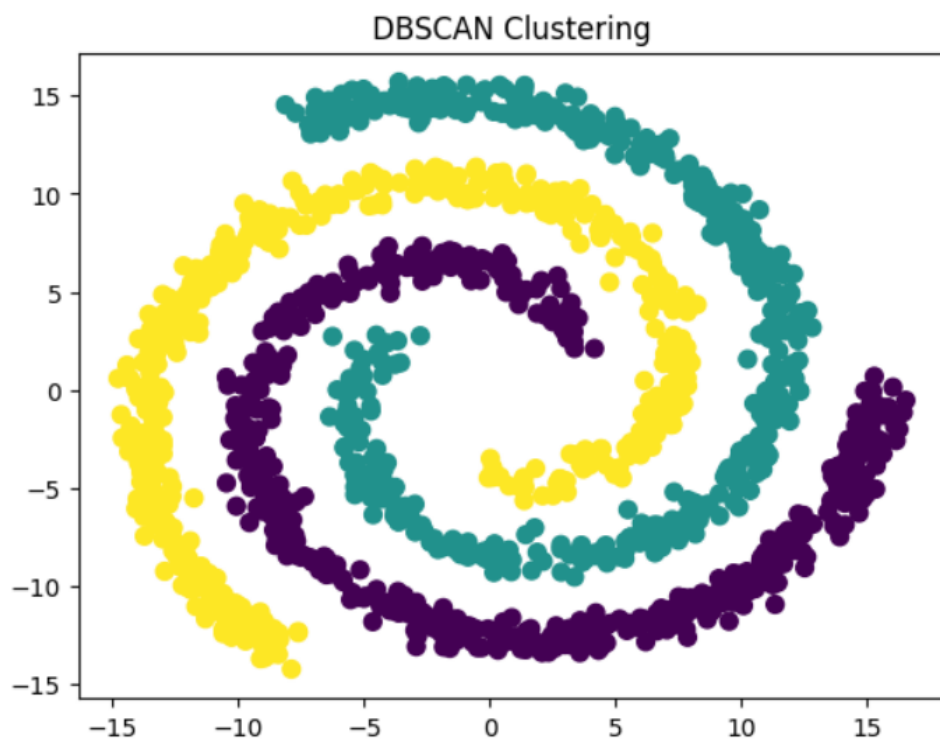


Рисунок 1.2.7 – DBSCAN для набора Spirals

В результате работы алгоритм Gaussian Mixtures для всех наборов данных, как и К-средних, получил плохой Rand Index (Рис. 1.2.8, 1.2.9, 1.2.10).

Rand Index для Gaussian Mixtures: -0.0005380546263830376

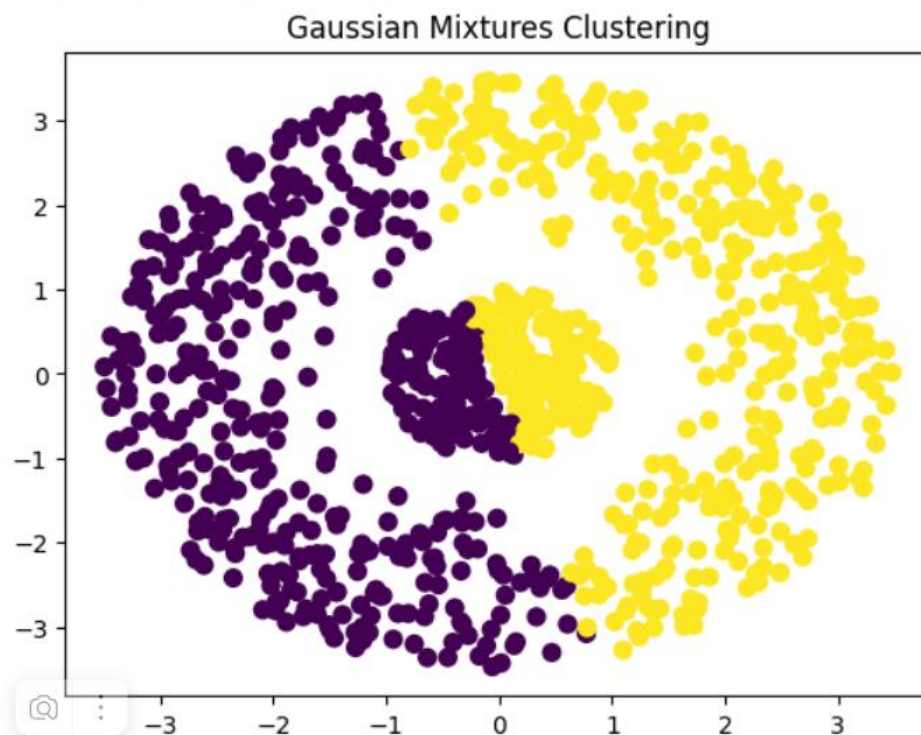


Рисунок 1.2.8 – Gaussian Mixtures для набора Circles

Rand Index для Gaussian Mixtures: 0.45048276305220886

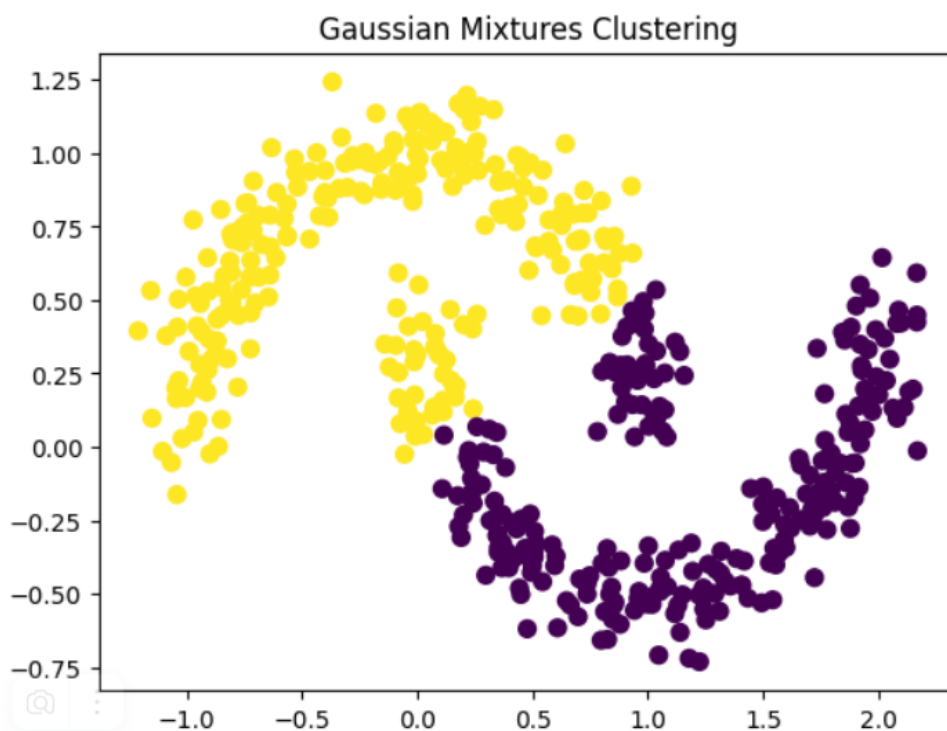


Рисунок 1.2.9 – Gaussian Mixtures для набора Moons

Rand Index для Gaussian Mixtures: 0.10345821064762306

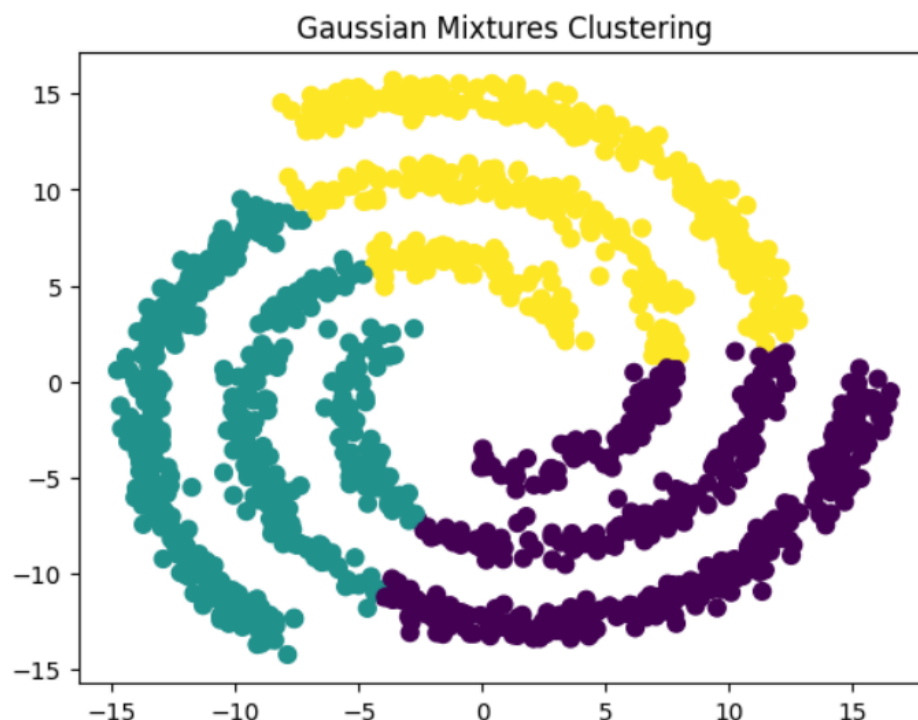


Рисунок 1.2.10 – Gaussian Mixtures для набора Spirals

Получив результаты кластеризации каждого метода и значения достигнутого качества, можно сделать вывод о том, что для задачи кластеризации наборов данных, состоящих из кластеров произвольной формы разных размеров, алгоритм DBSCAN показывает лучшие результаты. Полный код программы представлен в Приложении В.

1.3 Радиально-базисная нейронная сеть

Радиально-базисная нейронная сеть (RBFN) — это тип искусственной нейронной сети, которая использует радиально-базисные функции в качестве функций активации.

RBFN состоит из трех основных слоев:

1. Входной слой: принимает входные данные и передает их на скрытый слой.
2. Скрытый слой: содержит нейроны с радиально-базисными функциями, обычно гауссовыми функциями, в качестве функций активации. Каждый нейрон

в этом слое вычисляет расстояние между входным вектором и своим центром, а затем применяет радиально-базисную функцию к этому расстоянию.

3. Выходной слой: осуществляет линейную комбинацию выходов скрытого слоя для получения окончательного результата.

Принцип работы:

1. Обучение центроидов: центры радиально-базисных функций могут быть определены различными способами, например, методом k-средних или случайным выбором из входных данных.

2. Обучение весов: веса, связывающие скрытый и выходной слои, обычно обучаются методом наименьших квадратов для минимизации ошибки на обучающем наборе данных.

Рассмотрим процесс обучения RBNF наглядно. Средствами sklearn создадим набор данных (Moons) на плоскости с высоким показателем шума (Рис. 1.3.1).

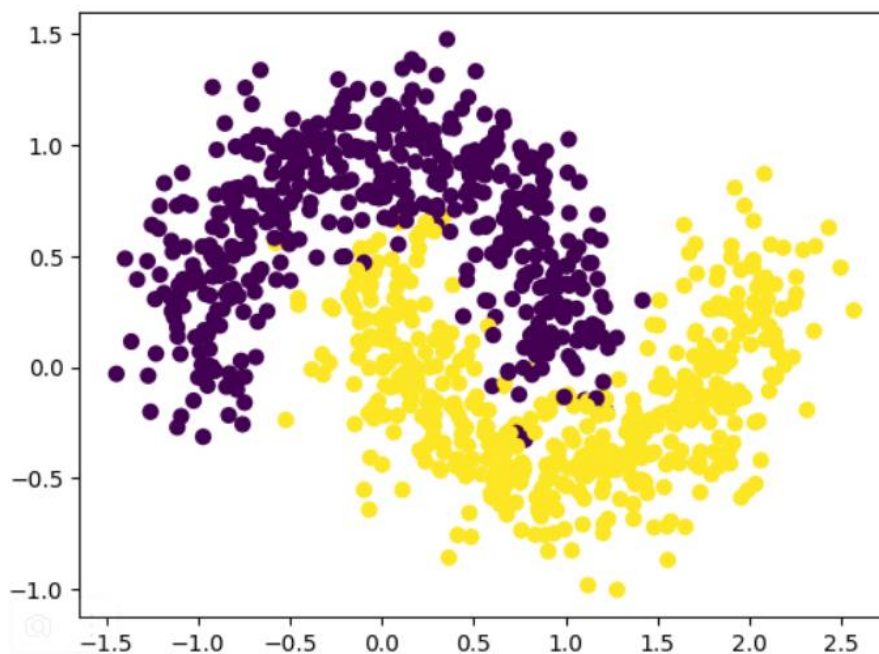


Рисунок 1.3.1 – Набор данных Moons

Реализуем двуслойную радиально-базисную сеть с автодифференцированием pytorch для классификации этих данных.

В ходе экспериментов с количеством нейронов, выявлено, что 20 нейронов является оптимальным количеством (Рис. 1.3.2).

Training RBF with 20 neurons:

```
Epoch 0, Train Loss: 0.7828121781349182, Test Loss: 0.7662612199783325
Epoch 100, Train Loss: 0.2601073384284973, Test Loss: 0.25169888138771057
Epoch 200, Train Loss: 0.15732058882713318, Test Loss: 0.13204990327358246
Epoch 300, Train Loss: 0.1119547113776207, Test Loss: 0.08318965882062912
Epoch 400, Train Loss: 0.09746095538139343, Test Loss: 0.06762730330228806
Epoch 500, Train Loss: 0.08933050185441971, Test Loss: 0.05983184650540352
Epoch 600, Train Loss: 0.08219616860151291, Test Loss: 0.05510370433330536
Epoch 700, Train Loss: 0.07705570757389069, Test Loss: 0.05016393959522247
Epoch 800, Train Loss: 0.07401622086763382, Test Loss: 0.04769500344991684
Epoch 900, Train Loss: 0.07179286330938339, Test Loss: 0.04596208408474922
Epoch 999, Train Loss: 0.07008248567581177, Test Loss: 0.04479524493217468
```

Рисунок 1.3.2 – Процесс обучения RBFN с 20 нейронами на 1000 эпохах

Для лучших найденных гиперпараметров отобразим анимацию процесса обучения, рисуя поверх графика данных центры и ширины RBF-нейронов. Анимация разделена на три части и представлена на Рисунках 1.3.3, 1.3.4 и 1.3.5.

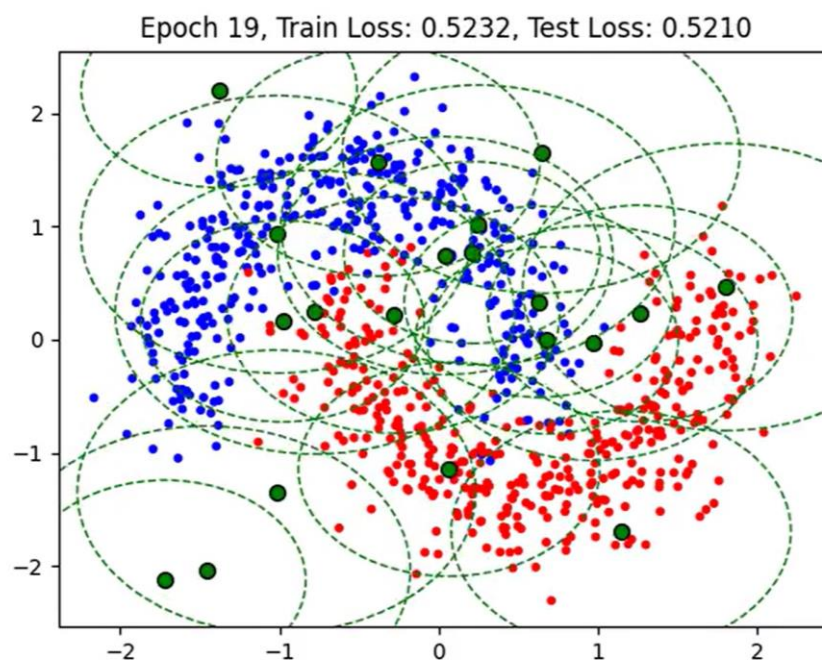


Рисунок 1.3.3 – Начало обучения RBFN

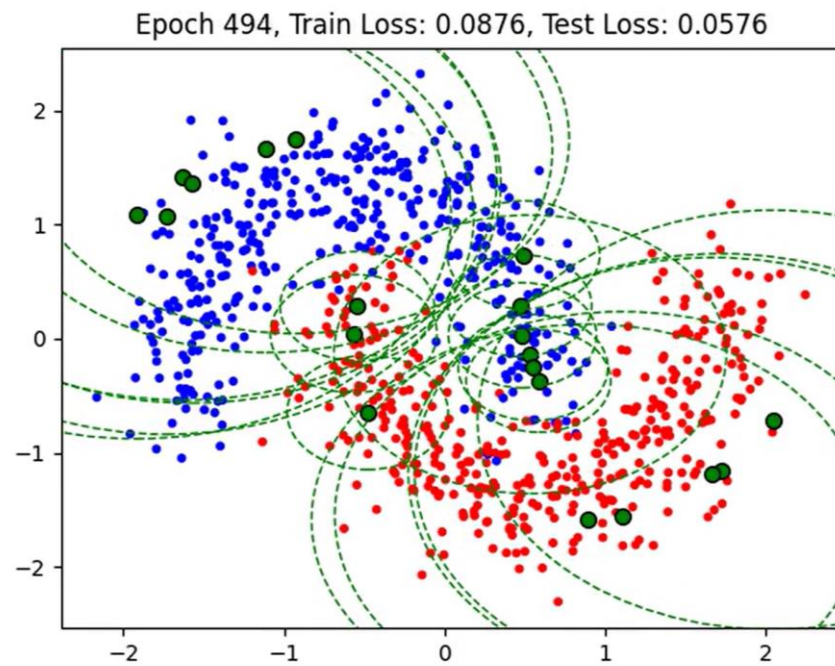


Рисунок 1.3.4 – Середина обучения RBFN

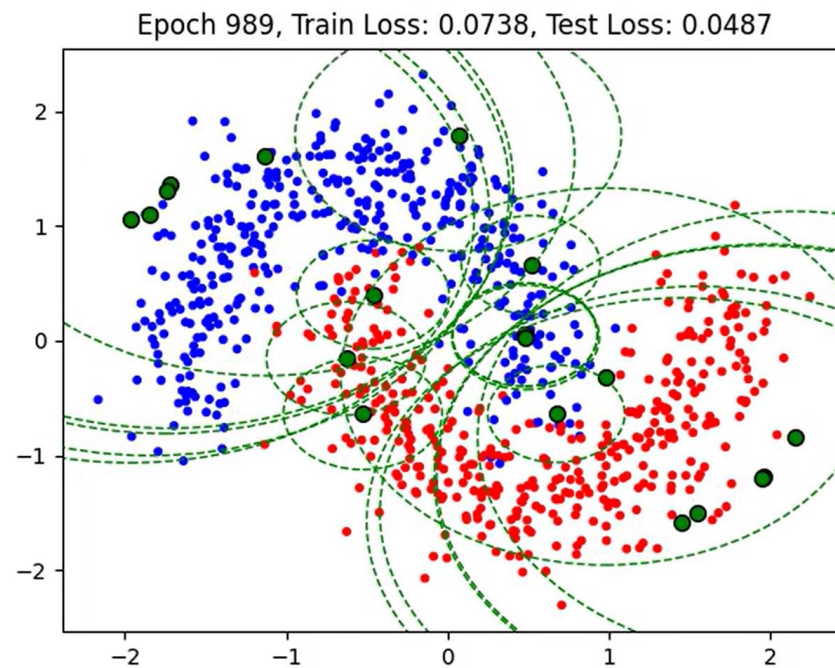


Рисунок 1.3.5 – Окончание обучения RBFN

В ходе анимации наблюдается как перемещаются нейроны и как изменяются их радиусы.

Таким образом, мы выяснили, что радиально-базисная нейронная сеть способна выполнять кластеризацию наборов данных, состоящих из кластеров произвольной формы, с произвольной точностью при достаточном количестве нейронов. Полный код программы представлен в Приложении С.

2 НЕЙРОСЕТЕВЫЕ МОДЕЛИ АНАЛИЗА ДАННЫХ

Нейросетевые модели анализа данных являются мощными инструментами, которые применяются для обработки и интерпретации большого объема информации.

Нейронные сети обработки изображений, такие как свёрточные нейронные сети (CNN), предназначены для анализа визуальных данных. Они эффективно выделяют иерархические признаки из изображений благодаря своей структуре, включающей свёрточные и пулинговые слои.

Нейронные сети обработки последовательностей, такие как рекуррентные нейронные сети (RNN) и их усовершенствованные версии. Эти сети способны учитывать зависимость между элементами последовательности, что делает их незаменимыми в задачах обработки естественного языка, машинного перевода, анализа временных рядов и генерации текста.

2.1 Векторные представления изображений

Векторное представление изображений (Img2Vec) — это метод, позволяющий преобразовать изображение в числовой вектор фиксированной длины. Этот вектор представляет важные характеристики изображения и может использоваться в различных задачах машинного обучения.

Основные этапы преобразования изображения в вектор включают:

1. Предобработка: Подготовка изображения, например, изменение размера, нормализация цветов и удаление шума.

2. Извлечение признаков: Использование моделей глубокого обучения, таких как свёрточные нейронные сети (CNN), для извлечения высокоуровневых признаков из изображения.

3. Построение вектора: Преобразование извлеченных признаков в вектор фиксированной длины. Обычно это делается путем использования выходов одного из последних слоев нейронной сети перед слоем классификации.

Перейдем к применению метода `Img2Vec`. Распакуем архив со всеми необходимыми файлами. Здесь находится `full_emoji.csv`, содержащий таблицу с названием, номером (из него можно получить путь к файлу), `unicode` кодом и изображением эмодзи (несколько для разных компаний). Также в архиве есть сами изображения эмодзи. Будем работать только с эмодзи от Google.

Получим векторные представления для изображений эмодзи с названиями 'grinning face', 'beaming face with smiling eyes' и 'flag: Scotland'. Сравним их между собой по косинусной похожести (Рис. 2.1.1).

Похожесть		и		-->	0.8933
Похожесть		и		-->	0.5992
Похожесть		и		-->	0.6007

Рисунок 2.1.1 – Сравнение эмодзи между собой

Из Рисунка 2.1.1 видно, что похожесть визуально похожих эмодзи получилась существенно больше, чем визуально не похожих.

Таким образом, метод векторного представления изображений `Img2Vec` показал высокую эффективность в сравнении эмодзи (картинок) и определении их похожести. Полный код программы представлен в Приложении D.

2.2 Векторные представления слов

Векторное представление слов (`Word2Vec`) — это метод, используемый в обработке естественного языка для представления слов и словосочетаний в виде числовых векторов. Эти векторы представляют семантические и контекстные отношения между словами, позволяя компьютеру понимать и анализировать текст на уровне, близком к человеческому.

Основная идея заключается в том, что слова, которые часто встречаются в схожих контекстах, имеют похожие значения. Например, слова «большой»,

«огромный» и «гигантский» могут быть представлены как близкие точки в векторном пространстве. Это позволяет моделировать отношения между словами и использовать их для различных задач, таких как классификация текста, машинный перевод и поиск информации.

Перейдем к тестированию технологии Word2Vec. Для этого загрузим корпус текстов, содержащих предложения как со словами, так и эмодзи.

Разделим предложения по словам, считая эмодзи отдельным словом. В корпусе ровно 1 000 000 строк. Эмодзи представлены в виде символов UNICODE с определенными кодами. Сделаем функцию `is_emoji(word)`, которая позволяет проверить, является ли символ или слово эмодзи. Средствами библиотеки `gensim` создадим и обучим Word2Vec модель на этом наборе слов и словарь всех "слов". Размер векторных представлений - 300.

Выполним предобработку корпуса средствами обученной Word2Vec модели и выведем на экран первые 10 элементов словаря, а также таблицу эмодзи (Рис. 2.2.1).

Первые 10 слов в словаре: ['i', 'the', 'to', 'you', 'a', '😄', 'my', 'and', 'me', 'for']
Размер таблицы с эмодзи: (2031, 2)

	Emoji	Unicode
0	😄	[128514]
1	😢	[128557]
2	😄	[128525]
3		[65039]
4	😊	[128522]

Рисунок 2.2.1 – Предобработка корпуса

Перейдем к проведению экспериментов с обученной Word2Vec моделью. Посчитаем и выведем на экран 10 наиболее похожих токенов на эмодзи «😄» (Рис. 2.2.2). Среди них не оказалось слов, модель представила 10 других эмодзи, которые действительно схожи со сравниваемым.

10 наиболее похожих токенов на эмоджон 😊 :

😊	: 0.6368870735168457
😄	: 0.6236443519592285
😊	: 0.6139289736747742
😊	: 0.5822786092758179
😄	: 0.5709506869316101
😄	: 0.5504471063613892
😊	: 0.5375324487686157
😊	: 0.5367890000343323
👍	: 0.4917653501033783
😬	: 0.4725322127342224

Количество слов: 0
Количество эмоджионов: 10

Рисунок 2.2.2 – Похожесть эмоджионов

Теперь посчитаем и выведем на экран 10 наиболее похожих токенов на слово «cat» (Рис. 2.2.3). В выведенном списке оказались 9 слов и 1 эмоджон.

10 наиболее похожих токенов на слово 'cat':

dog:	0.7575443387031555
kitten:	0.6640806198120117
puppy:	0.645494282245636
🐱	: 0.6347382664680481
pup:	0.6104207038879395
tiny:	0.6028496623039246
kitty:	0.6004958152770996
pet:	0.5716091394424438
cats:	0.5573223829269409
blanket:	0.5550395846366882

Количество слов: 9
Количество эмоджионов: 1

Рисунок 2.2.3 – Похожесть слов на эмоджионы

Для найденного эмоджона «🐱» выведем на экран 5 самых близких токенов (Рис. 2.2.4).

5 наиболее похожих токенов на '🐱':

pussy:	0.6456880569458008
cat:	0.6347382068634033
kitty:	0.6014494299888611
🐱	: 0.5989879369735718
🐾	: 0.5867421627044678

Рисунок 2.2.4 – Похожесть токенов на эмоджон

Как видно из Рисунка 2.2.4, наиболее похожим токеном оказалось слово «pussy». В списке также есть другие эмодзи.

Попробуем найти ближайшее слово X подходящее под уравнение (1) и не совпадающее ни с одним из исходных токенов, входящих в уравнение.

$$\text{🐱} - \text{🐶} = \text{cat} - X \quad (1)$$

Выведем на экран это слово и уровень похожести (Рис 2.2.5).

```
🐱 - 🐶 = cat - X
X = pussy
Уровень похожести: 0.6753081679344177
```

Рисунок 2.2.5 – Отношения эмодзи и слов

Составим другое уравнение (2), где X теперь будет эмодзи.

$$\text{cat} - \text{dog} = \text{🐱} - X \quad (2)$$

Выведем на экран искомый эмодзи и уровень похожести (Рис 2.2.6).

```
cat - dog = 🐱 - X
X = 🐶
Уровень похожести: 0.5843919515609741
```

Рисунок 2.2.6 – Отношения слов и эмодзи

Исходя из результата, становится ясно, что модель справилась с задачей нахождения неизвестного токена (слова или эмодзи).

В заключение составим проекцию векторов эмодзи на плоскость. Для этого возьмем вектора эмодзи и переведем их на плоскость, пользуясь методами PCA и TSNE. Затем нарисуем на плоскости миниатюры изображений эмодзи, координаты этих изображений определяются полученными координатами векторов на плоскости (Рис. 2.2.7).

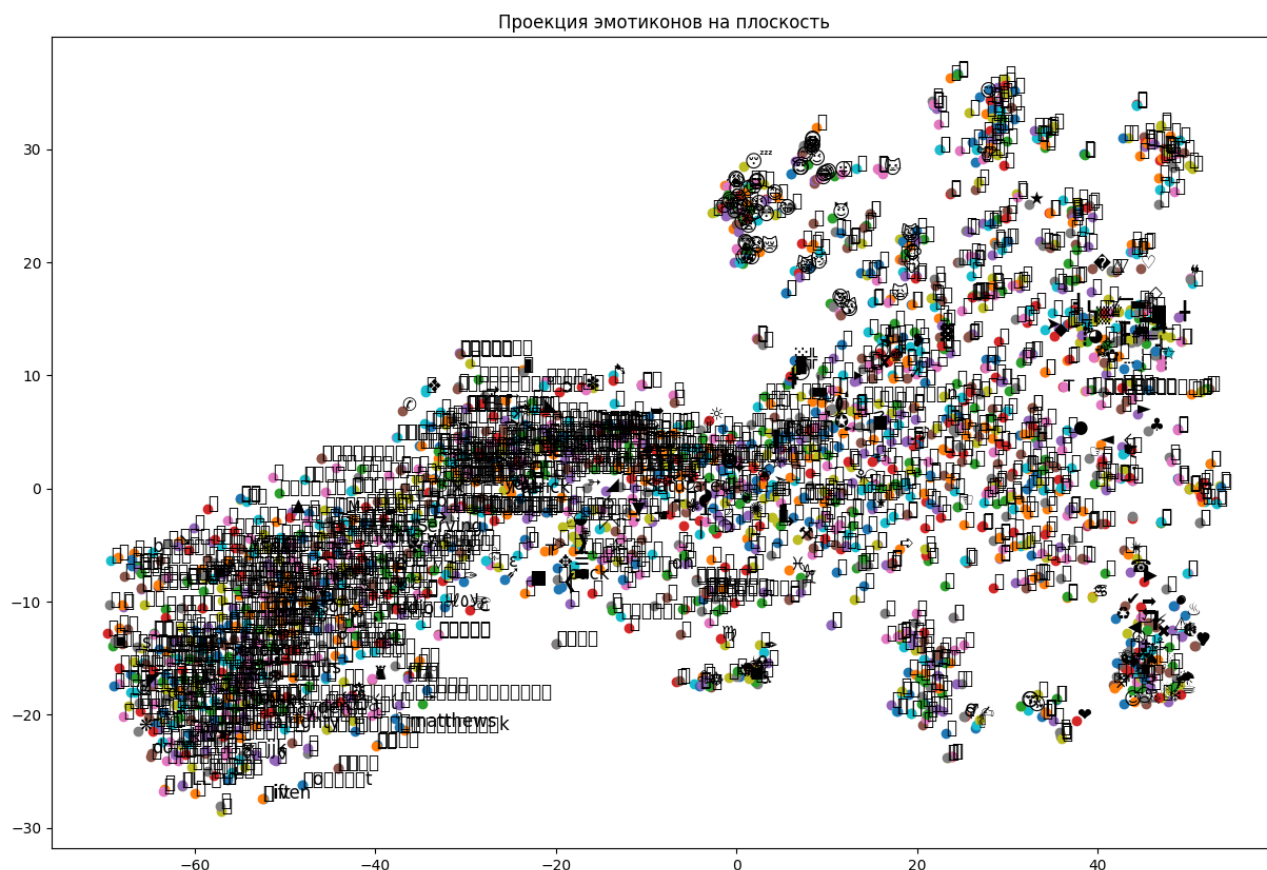


Рисунок 2.2.7 – Проекция эмодзи на плоскость

Из Рисунка 2.2.7 наблюдается группировка эмодзи на некоторые как мелкие кластеры, так и большие кластеры.

Таким образом, метод векторного представления слов Word2Vec способен улавливать сложные семантические отношения между словами и эмодзи, определять сходства и различия между ними. Полный код программы представлен в Приложении Е.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проектной работы были разработаны базовые модели машинного обучения для анализа данных, такие как логистическая регрессия, кластеризация, радиально-базисная нейронная сеть с автодифференцированием. Также были созданы нейросетевые модели анализа данных для векторного представления изображений и слов.

Были изучены теоретические основы каждой из моделей и алгоритмов, выбраны оптимальные параметры для их реализации, а также проведены эксперименты с различными наборами данных. В результате получены модели, которые могут быть использованы для решения прикладных задач искусственного интеллекта.

Таким образом, в ходе работы были достигнуты все поставленные цели и задачи. Разработанные модели могут найти применение в различных областях, таких как медицина, финансы, маркетинг и другие. Они могут использоваться для прогнозирования, классификации, кластеризации и других задач, требующих анализа больших объёмов данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Гудфеллоу, Я. Глубокое обучение / Я. Гудфеллоу, И. Бенджио, А. Курвилль, перевод с английского А. А. Слинкина. — 2-е изд. — Москва: ДМК Пресс, 2018. — 652 с. — ISBN 978-5-97060-618-6. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/book/107901>
2. Флах, П. Машинное обучение. Наука и искусство построения алгоритмов, которые извлекают знания из данных / П. Флах. — Москва: ДМК Пресс, 2015. — 400 с. — ISBN 978-5-97060-273-7. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/book/69955>
3. Барский, А. Б. Введение в нейронные сети: учебное пособие / А. Б. Барский. — 2-е изд. — Москва: ИНТУИТ, 2016. — 358 с. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/book/100684>
4. Ростовцев В.С. - Искусственные нейронные сети: учебник - Издательство "Лань" - 2019 - 216с. - ISBN: 978-5-8114-3768-9 - Текст электронный // ЭБС ЛАНЬ - URL: <https://e.lanbook.com/book/122180>
5. Хливненко Л.В., Пятакович Ф.А. - Практика нейросетевого моделирования: учебное пособие - Издательство "Лань" - 2019 - 200с. - ISBN: 978-5-8114-3639-2 - Текст электронный // ЭБС ЛАНЬ - URL: <https://e.lanbook.com/book/123697>

ПРИЛОЖЕНИЯ

Приложение А — Код реализации и тестирования логистической регрессии.

Приложение В — Код реализации и тестирования алгоритмов кластеризации.

Приложение С — Код реализации и тестирования радиально-базисной нейронной сети.

Приложение D — Код реализации и тестирования модели векторного представления изображений.

Приложение Е — Код реализации и тестирования модели векторного представления слов.

Приложение А

Код реализации и тестирования логистической регрессии на языке Python.

Листинг А – Код программы логистической регрессии

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, recall_score,
f1_score
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
import time
import pandas as pd
# Загрузите набор данных о рукописных цифрах MNIST из sklearn.
digits = load_digits()
X = digits.data
y = digits.target

# Изображения цифр представьте как 64-мерный вектор.
print(X.shape)
# Вывод первых 10 цифр
fig, axes = plt.subplots(nrows=2, ncols=5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(digits.images[i], cmap='gray')
    ax.set_title(f"Digit: {digits.target[i]}")
    ax.axis('off')

plt.show()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42) # разделение данных
logistic_model = LogisticRegression(max_iter=10000, n_jobs=-1,
random_state=42) # создание модели

logistic_model.fit(X_train, y_train) # обучение модели

y_pred = logistic_model.predict(X_test) # предсказания на тестовой выборке

# вычисление метрик
conf_matrix = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
recall = recall_score(y_test, y_pred, average='macro')
f1 = f1_score(y_test, y_pred, average='macro')
```

Продолжение Листинга А

```
print("Confusion Matrix:\n", conf_matrix)
print("Accuracy:", accuracy)
print("Recall:", recall)
print("F1 мера:", f1)

def calculate_pca_logistic(n_components, X_train, X_test, y_train, y_test):
    # Применение PCA и логистической регрессии
    pca = PCA(n_components=n_components)

    logistic_model = LogisticRegression(max_iter=10000, n_jobs=-1,
random_state=42)
    pipeline = Pipeline(steps=[('pca', pca), ('logistic', logistic_model)])

    start_time = time.time()
    pipeline.fit(X_train, y_train) # обучение модели
    end_time = time.time()
    training_time = end_time - start_time

    y_pred = pipeline.predict(X_test) # предсказания на тестовой выборке
    # вычисление метрик
    conf_matrix = confusion_matrix(y_test, y_pred)
    accuracy = accuracy_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred, average='macro')
    f1 = f1_score(y_test, y_pred, average='macro')

    return conf_matrix, accuracy, recall, f1, training_time

results = pd.DataFrame({
    'N': [],
    'Accuracy': [],
    'Recall': [],
    'F1_Score': [],
    'Training_Time': []
})
N = [2, 5, 10, 20, 30, 40, 50, 60]
for i in N:
    conf_matrix, accuracy, recall, f1, train_time = calculate_pca_logistic(i,
X_train, X_test, y_train, y_test)

    print(f'N = {i}:')
    print("Confusion Matrix:\n", conf_matrix)
    print("Accuracy:", accuracy)
    print("Recall:", recall)
    print("F1 мера:", f1)
    print("Train time:", train_time)
    print('-----')
```

Продолжение Листинга А

```
new_data = {
    'N': i,
    'Accuracy': accuracy,
    'Recall': recall,
    'F1_Score': f1,
    'Training_Time': train_time
}

results = results._append(new_data, ignore_index=True)

results
```

Приложение В

Код реализации и тестирования алгоритмов кластеризации на языке Python.

Листинг В – Код программы алгоритмов кластеризации

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans, DBSCAN
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score
import pandas as pd
import time

def generate_circles(inner_radius=1, outer_radius=3, inner_points=250,
outer_points=700, center=(0, 0)):
    theta_inner = np.random.rand(inner_points) * 2 * np.pi
    r_inner = np.sqrt(np.random.rand(inner_points)) * inner_radius
    x_inner = center[0] + r_inner * np.cos(theta_inner)
    y_inner = center[1] + r_inner * np.sin(theta_inner)
    inner_coords = np.column_stack((x_inner, y_inner))
    inner_labels = np.zeros(inner_points,)

    theta_outer = np.random.rand(outer_points) * 2 * np.pi
    r_outer = np.sqrt(np.random.rand(outer_points)) * (outer_radius -
inner_radius) + inner_radius + 0.5
    x_outer = center[0] + r_outer * np.cos(theta_outer)
    y_outer = center[1] + r_outer * np.sin(theta_outer)
    outer_coords = np.column_stack((x_outer, y_outer))
    outer_labels = np.ones(outer_points,)

    labels = np.concatenate((inner_labels, outer_labels))
    dataset = np.vstack((inner_coords, outer_coords))
    return dataset, labels

def generate_moons(n_points=500, noise=0.1, shuffle=True):
    dataset, labels = make_moons(n_samples=n_points, shuffle=shuffle,
noise=noise, random_state=42)
    return dataset, labels

def generate_spiral(n_points=500, n_spirals=3, noise=0.5, random_state=42):
    np.random.seed(random_state)
    theta = np.sqrt(np.random.rand(n_points)) * 2 * np.pi
    r_a = 2 * theta + np.pi # Radius
    data = []
    labels = []
```

```
    for i in range(n_spirals):
        x = r_a * np.cos(theta + 2 * np.pi * i / n_spirals) +
np.random.randn(n_points) * noise
        y = r_a * np.sin(theta + 2 * np.pi * i / n_spirals) +
np.random.randn(n_points) * noise
        data.extend(np.array([x, y]).T)
        labels.extend([i] * n_points)
    data = np.array(data)
    labels = np.array(labels)
    return data, labels

# Создание датасетов
circles, circles_labels = generate_circles()
moons, moons_labels = generate_moons()
spiral, spiral_labels = generate_spiral()

# Рисование датасетов
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.scatter(circles[:, 0], circles[:, 1], c=circles_labels, s=25)
plt.title('Circles')

plt.subplot(1, 3, 2)
plt.scatter(moons[:,0], moons[:,1], c=moons_labels, s=25)
plt.title('Moons')

plt.subplot(1, 3, 3)
plt.scatter(spiral[:, 0], spiral[:, 1], c=spiral_labels, s=25)
plt.title('Spirals')

plt.show()

# КЛАСТЕРИЗАЦИЯ ДЛЯ ПЕРВОГО НАБОРА ПАРАМЕТРОВ
X, y_true = circles, circles_labels

# Метод К-средних
kmeans = KMeans(n_clusters=2, random_state=0, max_iter=1000)
y_kmeans = kmeans.fit_predict(X)

# Метод DBSCAN

dbscan = DBSCAN(eps=0.5, n_jobs=-1, min_samples=2)
y_dbscan = dbscan.fit_predict(X)
```


Продолжение Листинга В

```
# Метод Gaussian Mixture

gmm = GaussianMixture(n_components=2, random_state=0, max_iter=1000)
y_gmm = gmm.fit_predict(X)

# Расчет Rand Index
ri_kmeans = adjusted_rand_score(y_true, y_kmeans)
ri_dbscan = adjusted_rand_score(y_true, y_dbscan)
ri_gmm = adjusted_rand_score(y_true, y_gmm)

def plot_clusters(X, y_pred, title):
    plt.scatter(X[:, 0], X[:, 1], c=y_pred, s=50, cmap='viridis')
    plt.title(title)
    plt.show()

print(f"Rand Index для K-means: {ri_kmeans}")
plot_clusters(X, y_kmeans, 'K-means Clustering')

print(f"Rand Index для DBSCAN: {ri_dbscan}")
plot_clusters(X, y_dbscan, 'DBSCAN Clustering')

print(f"Rand Index для Gaussian Mixtures: {ri_gmm}")
plot_clusters(X, y_gmm, 'Gaussian Mixtures Clustering')

circles_ri = [ri_kmeans, ri_dbscan, ri_gmm]

# КЛАСТЕРИЗАЦИЯ ДЛЯ ВТОРОГО НАБОРА ПАРАМЕТРОВ
X, y_true = moons, moons_labels

# Метод K-средних
kmeans = KMeans(n_clusters=2, random_state=0, max_iter=1000)
y_kmeans = kmeans.fit_predict(X)

# Метод DBSCAN
dbscan = DBSCAN(eps=0.2, n_jobs=-1, min_samples=3)
y_dbscan = dbscan.fit_predict(X)

# Метод Gaussian Mixture
gmm = GaussianMixture(n_components=2, random_state=0, max_iter=1000)
y_gmm = gmm.fit_predict(X)

# Расчет Rand Index
ri_kmeans = adjusted_rand_score(y_true, y_kmeans)
ri_dbscan = adjusted_rand_score(y_true, y_dbscan)
ri_gmm = adjusted_rand_score(y_true, y_gmm)
```

Продолжение Листинга В

```
print(f"Rand Index для K-means: {ri_kmeans}")
plot_clusters(X, y_kmeans, 'K-means Clustering')

print(f"Rand Index для DBSCAN: {ri_dbscan}")
plot_clusters(X, y_dbscan, 'DBSCAN Clustering')

print(f"Rand Index для Gaussian Mixtures: {ri_gmm}")
plot_clusters(X, y_gmm, 'Gaussian Mixtures Clustering')
moons_ri = [ri_kmeans, ri_dbscan, ri_gmm]

# КЛАСТЕРИЗАЦИЯ ДЛЯ ТРЕТЬЕГО НАБОРА ПАРАМЕТРОВ
X, y_true = spiral, spiral_labels

# Метод К-средних
kmeans = KMeans(n_clusters=3, random_state=0, max_iter=1000)
y_kmeans = kmeans.fit_predict(X)

# Метод DBSCAN
dbscan = DBSCAN(eps=1.3, n_jobs=-1, min_samples=5)
y_dbscan = dbscan.fit_predict(X)

# Метод Gaussian Mixture
gmm = GaussianMixture(n_components=3, random_state=0, max_iter=1000)
y_gmm = gmm.fit_predict(X)

# Расчет Rand Index
ri_kmeans = adjusted_rand_score(y_true, y_kmeans)
ri_dbscan = adjusted_rand_score(y_true, y_dbscan)
ri_gmm = adjusted_rand_score(y_true, y_gmm)

print(f"Rand Index для K-means: {ri_kmeans}")
plot_clusters(X, y_kmeans, 'K-means Clustering')
print(f"Rand Index для DBSCAN: {ri_dbscan}")
plot_clusters(X, y_dbscan, 'DBSCAN Clustering')
print(f"Rand Index для Gaussian Mixtures: {ri_gmm}")
plot_clusters(X, y_gmm, 'Gaussian Mixtures Clustering')
spiral_ri = [ri_kmeans, ri_dbscan, ri_gmm]

clustering_results = pd.DataFrame({
    'Circles': circles_ri,
    'Moons': moons_ri,
    'Spiral': spiral_ri
})
clustering_results.index = ['K-Means', 'DBSCAN', 'Gaussian Mixtures']
clustering_results
```

Приложение С

Код реализации и тестирования радиально-базисной нейронной сети на языке Python.

Листинг С – Код программы радиально-базисной нейронной сети

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.animation as animation
from IPython.display import HTML

# Создание данных
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Нормализация данных
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Преобразование данных в тензоры pytorch
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

# Отображение данных
plt.scatter(X[:, 0], X[:, 1], c=y, s=40)

class RBF(nn.Module): # Класс RBFN модели
    def __init__(self, input_dim, num_neurons):
        super(RBF, self).__init__()
        self.num_neurons = num_neurons
        self.centers = nn.Parameter(torch.randn(num_neurons, input_dim))
        self.log_widths = nn.Parameter(torch.zeros(num_neurons))
        self.linear = nn.Linear(num_neurons, 1)

    def rbf_function(self, x, centers, log_widths):
        # Расчет RBF-функций
```

```
        widths = torch.exp(log_widths)
        diff = x.unsqueeze(1) - centers.unsqueeze(0)
        dist_sq = torch.sum(diff**2, dim=2)
        return torch.exp(-dist_sq / (2 * widths**2))

    def forward(self, x):
        phi = self.rbf_function(x, self.centers, self.log_widths)
        return torch.sigmoid(self.linear(phi))

def train_rbf(model, X_train, y_train, X_test, y_test, epochs=1000,
lr=0.01):
    criterion = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    train_losses = []
    test_losses = []

    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()
        output = model(X_train)
        loss = criterion(output, y_train)
        loss.backward()
        optimizer.step()

        train_losses.append(loss.item())

    model.eval()
    with torch.no_grad():
        test_output = model(X_test)
        test_loss = criterion(test_output, y_test)
        test_losses.append(test_loss.item())

    if epoch % 100 == 0 or epoch == 999:
        print(f"Epoch {epoch}, Train Loss: {loss.item()}, Test Loss:
{test_loss.item()}")

    return train_losses, test_losses

neurons = [5, 10, 15, 20, 25, 30, 50]
results = {}

for num_neurons in neurons:
    print(f"\nTraining RBF with {num_neurons} neurons:")
    model = RBF(input_dim=2, num_neurons=num_neurons)
    train_losses, test_losses = train_rbf(model, X_train_tensor,
y_train_tensor, X_test_tensor, y_test_tensor, epochs=1000, lr=0.01)
    results[num_neurons] = (train_losses, test_losses)
```

Продолжение Листинга С

```
def animate_rbf_training(model, X_train, y_train, X_test, y_test,
epochs=500, lr=0.01, interval=50):
    fig, ax = plt.subplots()
    colors = ['blue' if label == 0 else 'red' for label in y_train]
    ax.scatter(X_train[:, 0], X_train[:, 1], c=colors, s=10)

    centers_scatter = ax.scatter([], [], c='green', s=50, edgecolor='k')
    radii_scatter = [plt.Circle((0, 0), 0, color='green', fill=False,
linestyle='--') for _ in range(model.num_neurons)]
    for circle in radii_scatter:
        ax.add_patch(circle)

    criterion = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
    train_losses = []
    test_losses = []

    def update(frame):
        model.train()
        optimizer.zero_grad()
        output = model(X_train)
        loss = criterion(output, y_train)
        loss.backward()
        optimizer.step()

        train_losses.append(loss.item())

        model.eval()
        with torch.no_grad():
            test_output = model(X_test)
            test_loss = criterion(test_output, y_test)
            test_losses.append(test_loss.item())

        ax.set_title(f"Epoch {frame}, Train Loss: {loss.item():.4f}, Test
Loss: {test_loss.item():.4f}")

        centers = model.centers.detach().numpy()
        widths = torch.exp(model.log_widths).detach().numpy()
        centers_scatter.set_offsets(centers)

        artists = [centers_scatter]
        for i, circle in enumerate(radii_scatter):
            circle.set_center(centers[i])
            circle.set_radius(widths[i])
            artists.append(circle)

    return artists
```

Продолжение Листинга С

```
    anim = animation.FuncAnimation(fig, update, frames=range(epochs),
interval=interval, blit=True, repeat=False)
    plt.close(fig)
    return anim

best_num_neurons = 20
best_model = RBF(input_dim=2, num_neurons=best_num_neurons)
anim = animate_rbf_training(best_model, X_train_tensor, y_train_tensor,
X_test_tensor, y_test_tensor, epochs=1000, lr=0.01, interval=50)
anim.save('rbf_training_animation.mp4', writer='ffmpeg') # Сохранение видео
# ВРЕМЯ ВЫПОЛНЕНИЯ ~2 МИН 15 СЕК

HTML('<video controls><source src="rbf_training_animation.mp4"
type="video/mp4"></video>') # Отображение в колабе (не работает)
```

Приложение D

Код реализации и тестирования модели векторного представления изображений на языке Python.

Листинг D – Код программы векторного представления изображений

```
!pip install img2vec_pytorch

import zipfile
import pandas as pd
import os
from PIL import Image
import torch
from img2vec_pytorch import Img2Vec
from torchvision import transforms
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
import warnings
warnings.filterwarnings('ignore')

import os
from time import time
import gdown
URL='https://drive.google.com/uc?id=1ONrKzVseOn8ZjBSK28VCh0bDYDrbun4k'
gdown.download(url=URL)
!unzip archive.zip

# Функция для получения векторного представления изображения
def get_image_vector(img_path, img2vec):
    img = Image.open(img_path).convert('RGB')
    return img2vec.get_vec(img, tensor=True)

# Функция для вычисления косинусной похожести
def cosine_similarity_vectors(vec1, vec2):
    return cosine_similarity(vec1.reshape(1, -1), vec2.reshape(1, -1))[0][0]

# Основная функция для сравнения эмодиконов
def compare_emoji(name_list):
    df = pd.read_csv('full_emoji.csv')
    df_filtered = df[df['name'].isin(name_list)]
    img2vec = Img2Vec(cuda=torch.cuda.is_available())
    vectors = {}
    # Получение векторных представлений для эмодиконов
    for _, row in df_filtered.iterrows():
        img_path = os.path.join('image/Google', f"{row['#']}.png")
        vec = get_image_vector(img_path, img2vec)
```

Продолжение Листинга D

```
        vectors[row['name']] = vec

# Сравнение эмодиконов между собой
for i in range(len(name_list)):
    for j in range(i + 1, len(name_list)):
        name1, name2 = name_list[i], name_list[j]
        vec1, vec2 = vectors[name1], vectors[name2]
        similarity = cosine_similarity_vectors(vec1, vec2)
        print(f"Похожесть {df_filtered[df_filtered['name'] ==
name1]['emoji'].values[0]} и {df_filtered[df_filtered['name'] ==
name2]['emoji'].values[0]} --> {similarity:.4f}")

name_list = ['grinning face', 'beaming face with smiling eyes', 'flag:
Scotland']
compare_emoji(name_list)
```


Приложение Е

Код реализации и тестирования модели векторного представления слов на языке Python.

Листинг Е – Код программы векторного представления слов

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import warnings
import re
import time
from gensim.models import Word2Vec
import pandas as pd
import os
import gdown
URL='https://drive.google.com/u/1/uc?id=1TPaQTYDpf7p2Ek1UkLdjK9T5ATztj7oF&export=download'
gdown.download(url=URL)
!unzip corpus.zip

# Определим функцию для проверки эмодзи
def is_emoji(word):
    return any('\U0001F600' <= char <= '\U0001F64F' or
              '\U0001F300' <= char <= '\U0001F5FF' or
              '\U0001F680' <= char <= '\U0001F6FF' or
              '\U0001F700' <= char <= '\U0001F77F' or
              '\U0001F780' <= char <= '\U0001F7FF' or
              '\U0001F800' <= char <= '\U0001F8FF' or
              '\U0001F900' <= char <= '\U0001F9FF' or
              '\U0001FA00' <= char <= '\U0001FA6F' or
              '\U0001FA70' <= char <= '\U0001FAFF' or
              '\U00002702' <= char <= '\U000027B0' or
              '\U000024C2' <= char <= '\U0001F251'
              for char in word)

# Функция для чтения и предобработки корпуса
def preprocess_corpus(file_path):
    with open(file_path, 'r', encoding='utf-8') as f:
        lines = f.readlines()

    sentences = []
    for line in lines:
        words = re.split(r'\s+', line.strip())
        sentences.append(words)
```

```
        return sentences

# Чтение и предобработка корпуса
file_path = 'corpus.txt'
sentences = preprocess_corpus(file_path)

# Создание и обучение модели Word2Vec
start_time = time.time()
model = Word2Vec(sentences, vector_size=300, window=5, min_count=1,
workers=4)
end_time = time.time()
train_time = end_time - start_time

print(f"Время обучения: {train_time} секунд")

# Извлечение эмотиконов и их кодов
emoji_dict = {word: [ord(char) for char in word] for word in
model.wv.index_to_key if is_emoji(word)}

# Создание таблицы с эмотиконами и их кодами
emoji_df = pd.DataFrame([(word, codes) for word, codes in
emoji_dict.items()], columns=['Emoji', 'Unicode'])

vocabulary = list(model.wv.index_to_key)
print(f"Первые 10 слов в словаре: {vocabulary[:10]}")
print(f"Размер таблицы с эмотиконами: {emoji_df.shape}")
emoji_df.head()

# Найти 10 наиболее похожих токенов на эмотикон 😊
similar_tokens = model.wv.most_similar('😊', topn=10)
similar_words = [token for token, _ in similar_tokens if not
is_emoji(token)]
similar_emojis = [token for token, _ in similar_tokens if is_emoji(token)]

print("10 наиболее похожих токенов на эмотикон 😊:")
for token, similarity in similar_tokens:
    print(f"{token}: {similarity}")

print(f"\nКоличество слов: {len(similar_words)}")
print(f"Количество эмотиконов: {len(similar_emojis)}")

similar_tokens_cat = model.wv.most_similar('cat', topn=10)
similar_words_cat = [token for token, _ in similar_tokens_cat if not
is_emoji(token)]
```

```
similar_emojis_cat = [token for token, _ in similar_tokens_cat if
is_emoji(token)]

print("10 наиболее похожих токенов на слово 'cat':")
for token, similarity in similar_tokens_cat:
    print(f"{token}: {similarity}")

print(f"\nКоличество слов: {len(similar_words_cat)}")
print(f"Количество эмотиконов: {len(similar_emojis_cat)}\n")

similar_tokens_cat_emoji = model.wv.most_similar(similar_emojis_cat[0],
topn=5)
similar_words_cat_emoji = [token for token, _ in similar_tokens_cat_emoji if
not is_emoji(token)]
similar_emojis_cat_emoji = [token for token, _ in similar_tokens_cat_emoji
if is_emoji(token)]

print(f"5 наиболее похожих токенов на '{similar_emojis_cat[0]}':")
for token, similarity in similar_tokens_cat_emoji:
    print(f"{token}: {similarity}")

wv = model.wv
tokenA='🐱'
tokenB='🐶'
tokenC='cat'

out = wv[tokenC]-wv[tokenA]+wv[tokenB]
candidate = [x for x in wv.similar_by_vector(out,topn=50) if x[0] not in
[tokenA,tokenB,tokenC]]

res = [x for x in candidate if not is_emoji(x[0])][0]

print(f"🐱 - 🐶 = cat - X\nX = {res[0]}")
print(f"Уровень похожести: {res[1]}")

tokenA='cat'
tokenB='dog'
tokenC='🐱'

out = wv[tokenC] - wv[tokenA] + wv[tokenB]
candidate = [x for x in wv.similar_by_vector(out,topn=50) if x[0] not in
[tokenA,tokenB,tokenC]]
res = [x for x in candidate if is_emoji(x[0])][0]
print(f"cat - dog = 🐱 - X\nX = {res[0]}")
```

```
print(f"Уровень похожести: {res[1]}")

warnings.filterwarnings('ignore')

# Извлечение векторов эмотиконов из модели
emoji_vectors = [model.wv[word] for word in emoji_df['Emoji']]
emoji_labels = emoji_df['Emoji']

# Применение PCA для уменьшения размерности до 50
pca = PCA(n_components=50)
emoji_vectors_pca = pca.fit_transform(emoji_vectors)

# Применение TSNE для уменьшения размерности до 2
tsne = TSNE(n_components=2)
emoji_vectors_tsne = tsne.fit_transform(emoji_vectors_pca)

plt.figure(figsize=(15, 10))

for i, (x, y) in enumerate(emoji_vectors_tsne):
    plt.scatter(x, y)
    plt.text(x+0.01, y+0.01, emoji_labels[i], fontsize=12)

plt.title("Проекция эмотиконов на плоскость")
plt.show()
```